CO3133 · DEEP LEARNING AND ITS APPLICATIONS · SEMESTER-261

Assignment 2 · Deep Learning on Large-Scale Data and Specialized Tasks

Scratch U-Net vs. Pretrained DeepLabV3 for Semantic Segmentation on Cityscapes

19 classes per pixel · city-disjoint splits · one protocol for every run

GROUP RTX5090

Dang Thanh Phu · 2452967

Cao Duc Thanh · 2453164

INSTRUCTOR

Dr. Le Thanh Sach

HCMUT – VNU-HCM · Faculty of CSE

AGENDA

Two presenters, one storyline

PART A · CAO DUC THANH

1 · Problem formulation

2 · Cityscapes data and EDA

3 · Split, preprocessing and leakage control

4 · Pipeline and training setup

5 · U-Net, DeepLabV3, fine-tuning and losses

PART B · DANG THANH PHU

6 · Experimental design and main results

7 · Training behaviour, ablations and cost

8 · Error analysis and qualitative examples

9 · Short demo and reproducibility

10 · Conclusion, links and AI disclosure

1 · PROBLEM FORMULATION

Every pixel gets one of 19 classes

INPUT

`x : [3, 512, 1024]`

RGB frame, ImageNet-normalized

→

MODEL

`z = f(x) : [19, 512, 1024]`

one logit per class per pixel

→

OUTPUT

`ŷ = argmax z : [512, 1024]`

class id 0–18; void (255) is skipped

Prediction unit = the pixel. Loss: per-pixel cross-entropy on logits, nn.CrossEntropyLoss(ignore_index=255); metrics from one global confusion matrix: mIoU, mean Dice, pixel accuracy.

Class imbalance

Road, building and vegetation dominate; rare classes have few pixels

Thin objects

Poles, traffic lights and signs are only a few pixels wide

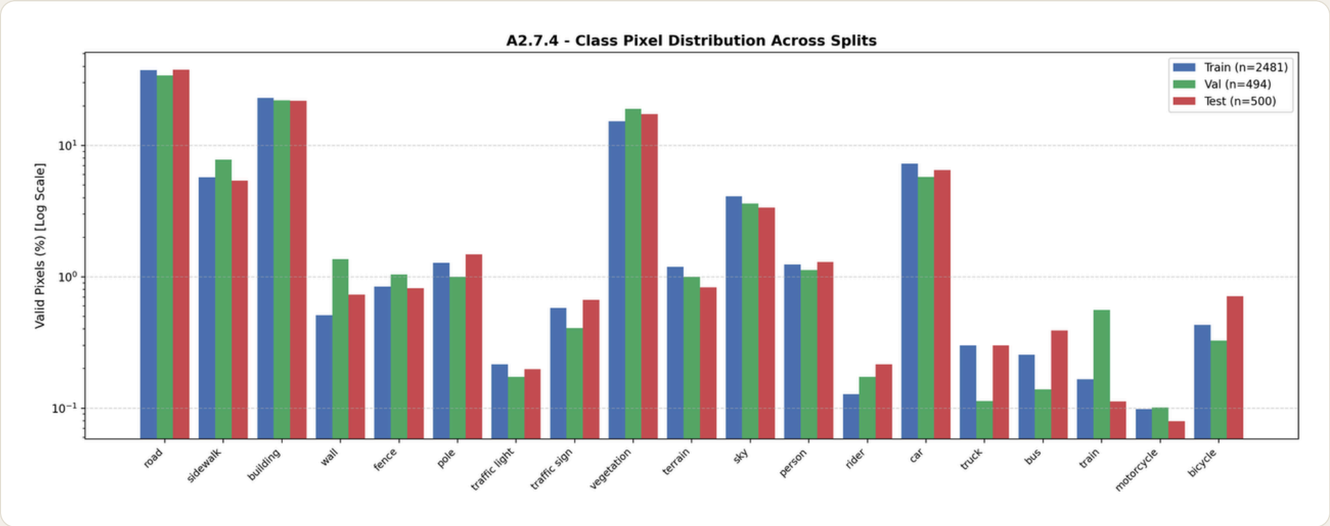
Look-alike neighbours

Road vs. sidewalk; building vs. wall vs. fence

New cities at test

Test cities never appear in training

A few classes own most of the pixels



Pixel share per class in train / val / test (src/eda.py)

Cityscapes *fine*: 34 raw IDs → 19 classes in 7 groups, the rest → void 255. Registration required; non-commercial licence.

5,000

fine-annotated frames, 50 cities, 2048×1024 PNG; 2,975 / 500 / 1,525 official split (test labels hidden)

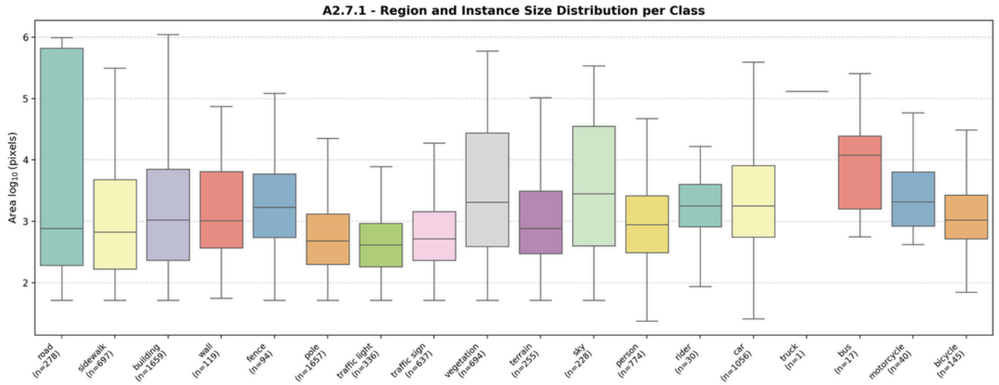
75.69%

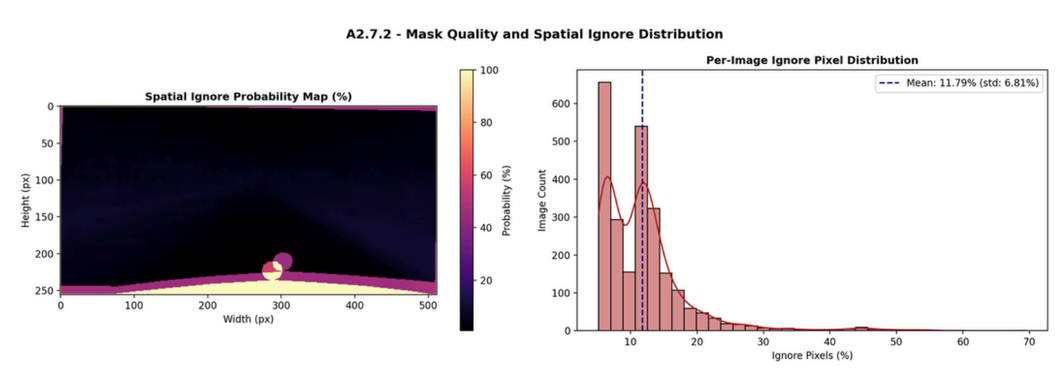
of training pixels are road (37.39%), building (22.99%) or vegetation (15.30%)

3.52%

for the 10 rarest classes together: 381 road pixels for every motorcycle pixel

Thin objects make tiny regions





92.96%

of pole regions are smaller than 1,024 px (32×32). Median size: pole 87, light 113, sign 137 vs. car 446 px

11.96%

of training pixels are void (val 9.08%, test 12.55%). The car hood at the bottom is always void

11.72

classes in one training image on average (min 5, max 18): busy street scenes

Split by city, not by image



Train = 14 cities of the official train set · **Val** = Erfurt, Jena, Tübingen, Zurich (model and loss choices) · **Test** = the official val set: Frankfurt 267, Lindau 59, Münster 174 (used once)

No near-duplicates

Frames of one city come from the same drives. Each city goes to one split, and the script checks that no city appears twice.

Labels stay valid

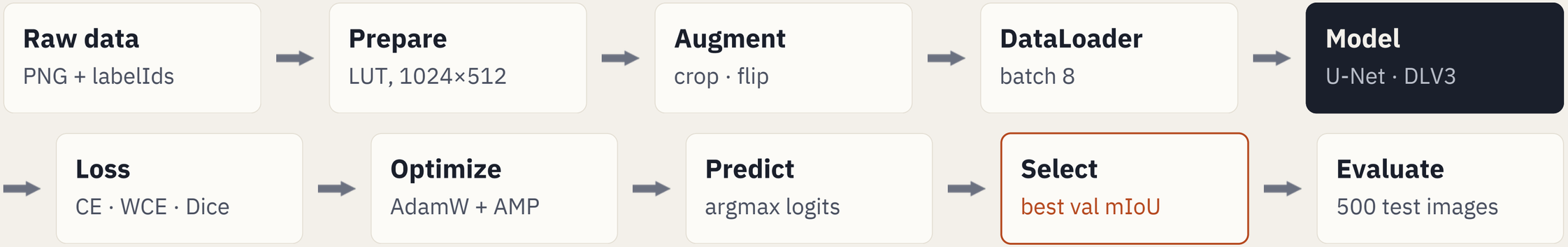
34 raw IDs → 19 trainIds by a 256-entry LUT. Resized once to 1024×512: bilinear images, nearest masks, so no new IDs.

Augment train only

Random 512×512 crop, flip p = 0.5, colour jitter 0.2, ImageNet mean/std. Val and test: full 1024×512, no augmentation.

```
checked batch → images float32 [8, 3, 512, 512] · masks int64 [8, 512, 512] · values {0…18, 255}
```

One pipeline shared by all six runs



Optimizer · scheduler

AdamW (wd 0.01) · none

LR · head / backbone

1e-4 / 1e-5 (full 1e-6) · U-Net 1e-3

Epochs · batch

U-Net 40 · DLV3 60 · batch 8

Regularization

augmentation + weight decay

Checkpoint · early stop

best val mIoU · none

Seed · precision

666 · fp16 AMP + GradScaler

Hardware

RTX 5070 Ti · Colab RTX PRO 6000 (E1)

Software

PyTorch 2.11 · torchvision 0.26

5 · MODELS

Scratch U-Net vs. pretrained DeepLabV3

FROM SCRATCH · OUR CODE

U-Net (baseline)

```
4 × [Conv3×3-BN-ReLU ×2 → MaxPool]
channels 32 → 64 → 128 → 256 → 512
4 × [ConvTranspose 2×2 + skip]
→ Conv1×1 → 19 logits
```

Skip connections keep fine detail; small and fast. No pretrained knowledge and a limited view of the scene. Kaiming init.

7,763,635

parameters · 193.7 GFLOPs per 1024×512 image

IMAGENET-1K BACKBONE · TORCHVISION

DeepLabV3-ResNet50

```
ResNet-50, dilated layer3/4 (stride 8)
ASPP: 1×1 + 3×3 rates 12 / 24 / 36
+ image pooling, 256 channels each
→ 1×1 → 19 logits → bilinear ×8
```

Reuses ImageNet features and sees context at many scales. No aux head; new 19-class classifier. ×8 upsampling blurs borders.

39,638,355

parameters · 654.3 GFLOPs (3.4× U-Net)

5 · FINE-TUNING DEPTH & LOSSES

Two knobs we turn on DeepLabV3

WHICH LAYERS TRAIN (E1)

Frozen · ASPP head only — **16.13M**

Partial · layer3 + layer4 + head — **38.19M**

Full · everything — **39.64M**

Trainable parameters. Head LR 1e-4 everywhere; backbone LR 1e-5 (partial) but 1e-6 (full).

WHICH LOSS (E2) · SRC/LOSSES.PY, OURS

CE

plain cross-entropy, label 255 ignored

Weighted CE

`w_c = 1 + log(max N / N_c)`

road 1.00 · rider 6.68 · motorcycle 6.94

CE + Dice

`L = CE + (1 − mean soft Dice), λ = 1`

Dice per class present in the batch, so small classes count as much as big ones

Ours: data pipeline, transforms, U-Net, losses, train/eval loops, confusion-matrix metrics, analysis scripts. **Libraries:** torchvision (DeepLabV3, ResNet-50, weights), cityscapesscripts, OpenCV, SciPy, matplotlib.

One changed factor per experiment

Every decision uses **validation**; the test set is used once per selected checkpoint. Metrics: mIoU (primary), mean Dice, pixel accuracy, rare-10 IoU, and cost.

Experiment	Changed factor	Held fixed	Not matched	Hypothesis → decision rule
E0 – Baseline vs. pretrained	U-Net from scratch vs. DeepLabV3-M1 (frozen) with ImageNet backbone	Data split, preprocessing (512×1024 resize, 512×512 crops), AdamW, CE loss, seed	Epochs (40 / 60), learning rate, batch size (8 / 16), GPU	DeepLabV3-M1 (frozen) beats U-Net by ≥ 5 mIoU points val mIoU
E1 – Fine-tuning depth	Trainable backbone: frozen (M1) / partial (M2) / full (M3)	DeepLabV3, CE loss, head LR 10^{-4}, 60 epochs, batch 16, GPU, seed, data	Backbone LR (partial 10^{-5}, full 10^{-6})	Choose m2-partial or m3-full if it is ≥ 2 mIoU points val mIoU above frozen
E2 – Loss variant	Loss on DeepLabV3-M2 (partial fine-tuning): CE vs. weighted CE vs. CE + Dice	DeepLabV3-M2 (partial: layer3 + layer4 + head), head LR 10^{-4}, backbone LR 10^{-5}, 60 epochs, seed, data	Batch size (CE: 16, others: 8), GPU (CE: RTX PRO 6000, others: RTX 5070 Ti)	Weighted CE or CE + Dice adds ≥ 1 mIoU points val mIoU over CE

6 · MAIN RESULTS · 494 VAL / 500 TEST IMAGES

Partial fine-tuning + CE + Dice leads

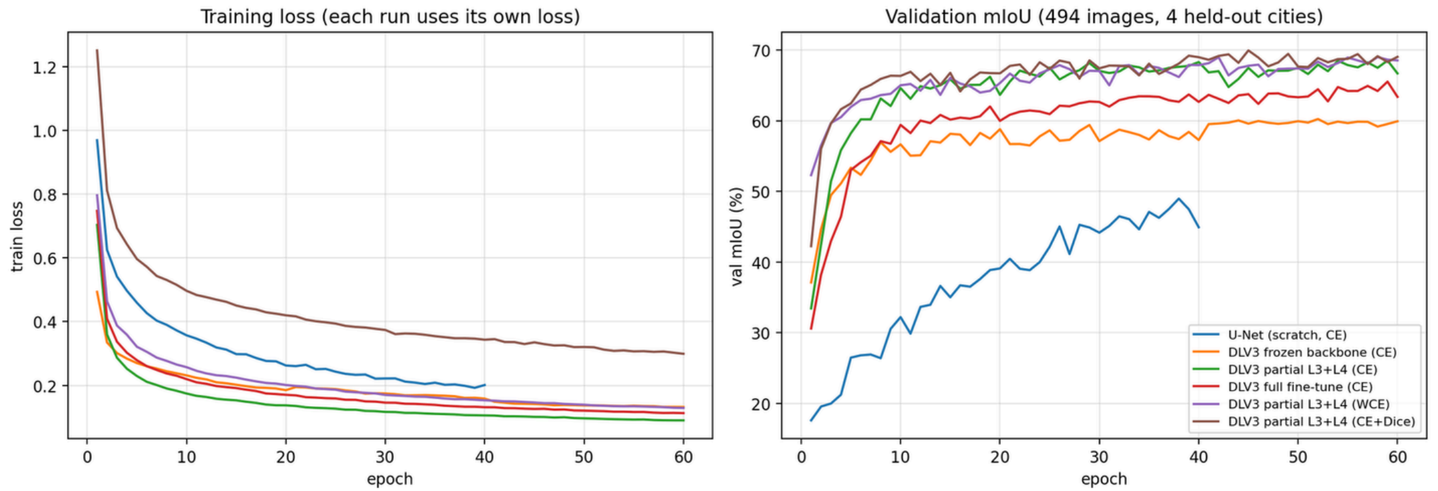
Validation · used for selection | Test · used once - only to show as one time result

		Validation				Test			
Run (model,loss)	Ep.	mIoU	Dice	PixAcc	Loss	mIoU	Dice	PixAcc	Loss
U-Net, CE	38	48.97	60.66	90.81	0.304	49.35	60.86	91.65	0.278
DLV3 frozen, CE	52	60.25	73.39	91.80	0.284	57.01	70.21	91.55	0.287
DLV3 full, CE	59	65.54	77.79	93.24	0.222	61.10	73.83	93.05	0.227
DLV3 partial, CE	59	68.54	80.17	94.29	0.197	66.53	78.48	94.07	0.201
DLV3 partial, WCE	58	69.02	80.63	93.99	0.339	67.21	79.10	93.90	0.344
DLV3 partial, CE+Dice	45	69.96	81.31	94.17	0.481	68.09	79.81	93.92	0.537

Sorry for the missing data in test, i will update this result in report as soon as possible in the report

All values in %. One run per setting (seed 666). The final model is picked by validation mIoU, not by test. Pixel accuracy is highest for partial CE (94.07%). E1 test numbers come from the Colab evaluation files.

Pretrained runs learn much faster



Each run uses its own loss, so the CE + Dice loss curve is on a different scale.

Fast start

Every DLV3 run passes 55% val mIoU by epoch 2–8. U-Net never does: best 48.97% at epoch 38 of 40.

Noisy, no scheduler

Last 20 epochs: DLV3 val mIoU moves 0.3–0.9 pp per epoch (median, max 2.6). U-Net: 1.4 pp (max 4.1).

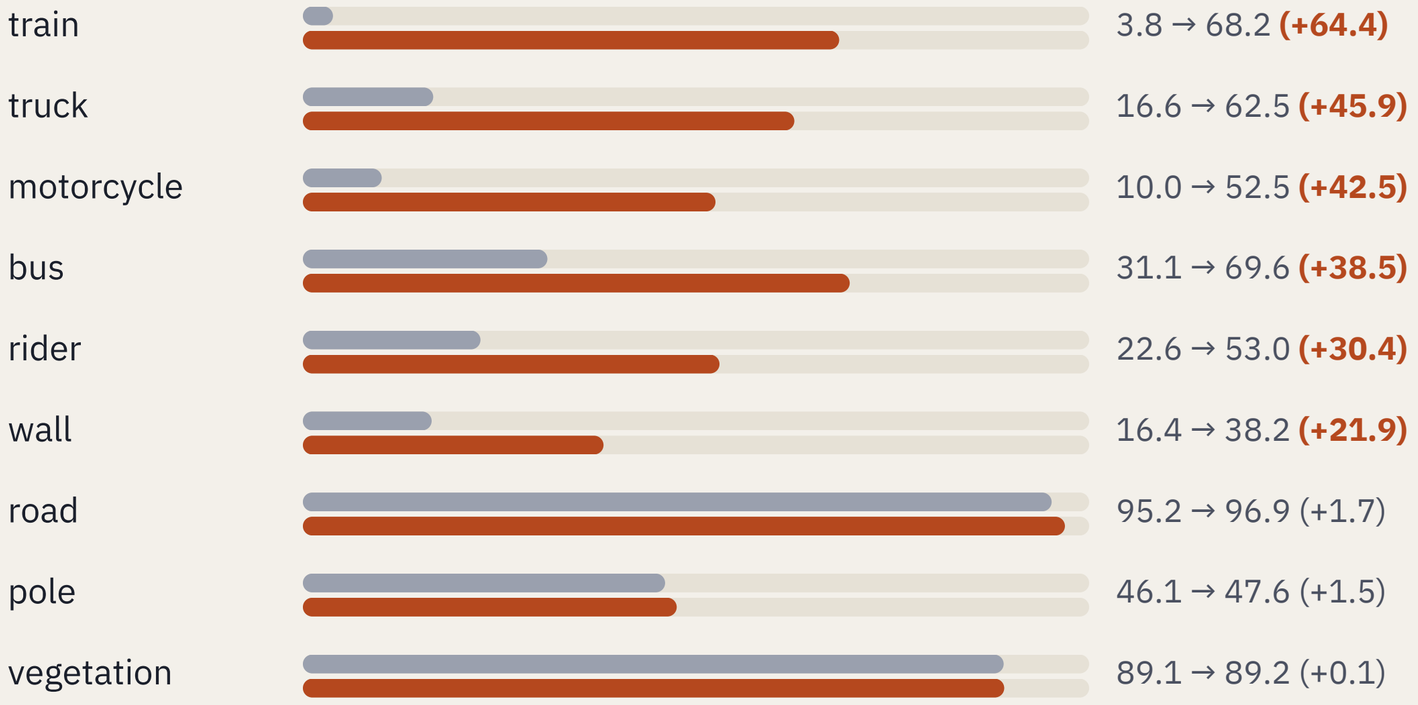
Mild overfit in loss only

Val loss bottoms at epoch 25–30, then rises a little; val mIoU does not drop. Partial and full peak at epoch 59 of 60.

7 · PER-CLASS TEST IOU · U-NET VS. FINAL DEEPLABV3

Gains come from big rare objects

Upper bar: **U-Net** · lower bar: **final DLV3 (partial, CE + Dice)** · bar length = test IoU, 0–100%



+18.74 pp
test mIoU, 49.35 → 68.09

+31.32 pp
rare-10 test IoU, 27.01 → 58.33

Thin poles and common classes barely move. Pretraining helps large objects with few training examples most.

7 · ABLATIONS · DECIDED ON VALIDATION

Depth matters; the loss matters less

E1 · FINE-TUNING DEPTH

+8.29 pp

partial over frozen in val mIoU, above the 2 pp rule → **partial chosen**

Variant	Trainable	Val mIoU	Test mIoU
Frozen	16.13M	60.25	57.01
Full	39.64M	65.54	61.10
Partial	38.19M	68.54	66.53

Full is in between, but it also used a 10× smaller backbone LR, so we cannot say “too many layers” is the cause.

E2 · LOSS ABLATION (PARTIAL DLV3)

+1.42pp

Rule met by CE + Dice

Loss	Val mIoU
CE	68.54
WCE	69.02
CE + Dice	69.96

Most of the CE + Dice test gain is one class: train +19.96 pp. One seed only, so treat it as noisy.

2.4× slower, still about 60 FPS

Model	GPU	Epochs	Train / epoch	Total train	Total val	Total
U-Net	RTX 5070 Ti	40	30.3 s	1,225 s	204 s	1,429 s
DLV3 frozen	RTX PRO 6000	60	14.4 s	873 s	198 s	1,071 s
DLV3 partial	RTX PRO 6000	60	22.6 s	1,356 s	198 s	1,554 s
DLV3 full	RTX PRO 6000	60	25.7 s	1,542 s	198 s	1,740 s
DLV3 partial + WCE	RTX 5070 Ti	60	51.7 s	3,123 s	450 s	3,573 s
DLV3 partial + CE+Dice	RTX 5070 Ti	60	54.9 s	3,311 s	492 s	3,803 s

2.4×
slower than U-Net: 16.5 vs. 7.0 ms median per image

≈ 60 FPS
at 1024×512 on one RTX 5070 Ti, fast enough for real time

+18.74 pp
test mIoU in return (final model vs. U-Net)

Latency: RTX 5070 Ti, fp16, batch 1, 3×512×1024, 200 timed runs. Train time = all epochs (U-Net 40 on 5070 Ti; DLV3 60 on Colab RTX PRO 6000), so GPUs differ.

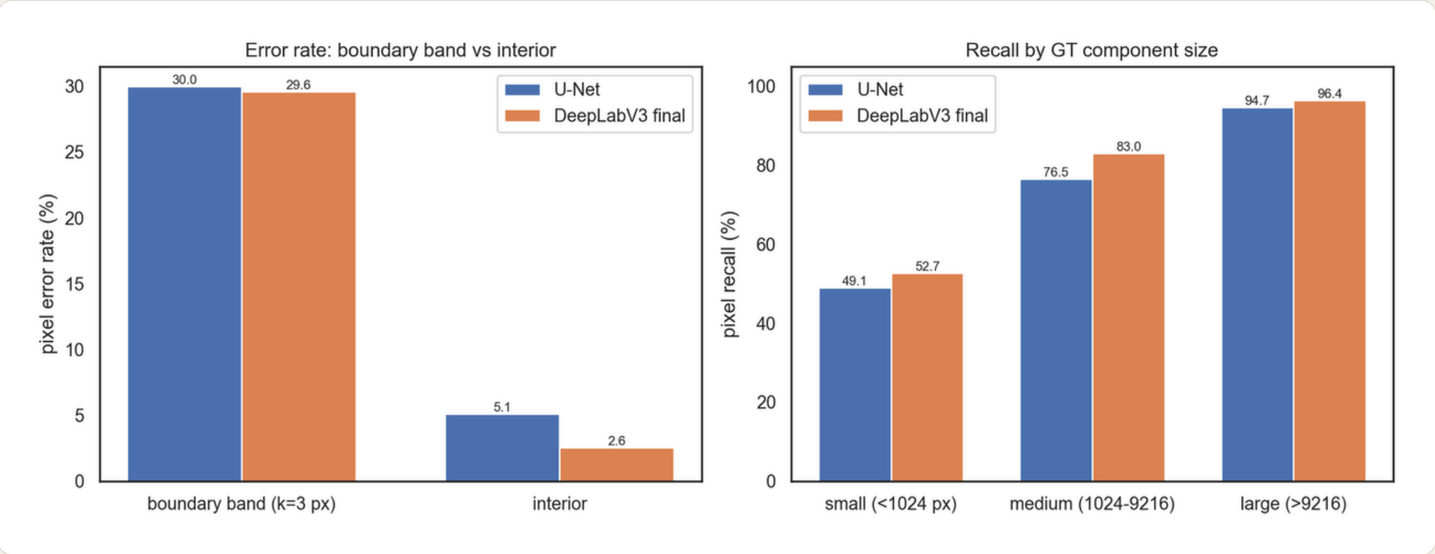
8 · ERROR ANALYSIS · FINAL MODEL, TEST SET

Most errors sit on object borders

Each of the 13.93M wrong test pixels goes to exactly one group, checked in order 1 → 4. Class imbalance is counted per class.

Error group	How often	Example image	Likely cause → improvement
1 · Boundary (≤ 3 px from a border)	63.3% of errors, from only 13.0% of pixels	frankfurt_000001_066574	Stride 8 + ×8 upsampling blurs edges → decoder with low-level features, boundary loss
2 · Small region (< 1,024 px)	2.4%; recall 52.7% small vs. 96.4% large	frankfurt_000001_028335	Detail lost when downsampled → larger input, multi-scale testing
3 · Related classes	19.2%; wall recall 43.9%, 26.1% → building	frankfurt_000001_017459	Look-alike texture, no 3D cue → more context, oversample wall / fence
4 · Other	15.1%; building ↔ vegetation alone 15.7%	lindau_000047	Trees over walls; Lindau looks unlike training cities → more varied data
Class imbalance	rare-10: 4.2% of pixels, 22.2% of errors	IoU 58.3% vs. 91.7% (top-3)	Few regions (train: 334) → rare-class sampling, more data, more seeds

Pretraining fixes interiors, not borders



Blue = U-Net, orange = final DeepLabV3 (test set)

29.6% vs. 2.56%

final error rate at borders vs. inside regions: 11.5× higher at borders

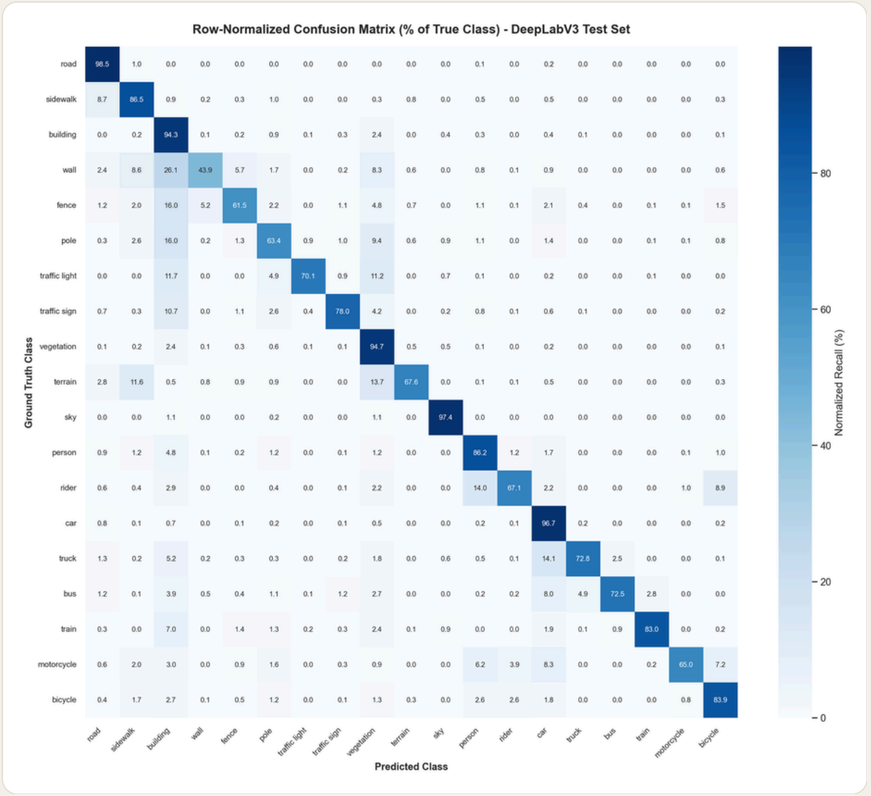
30.0% vs. 5.11%

for U-Net: DLV3 halves interior errors, but border errors barely move

52.7% → 96.4%

recall from small to large regions (U-Net 49.1% → 94.7%)

Top mix-ups are look-alike pairs

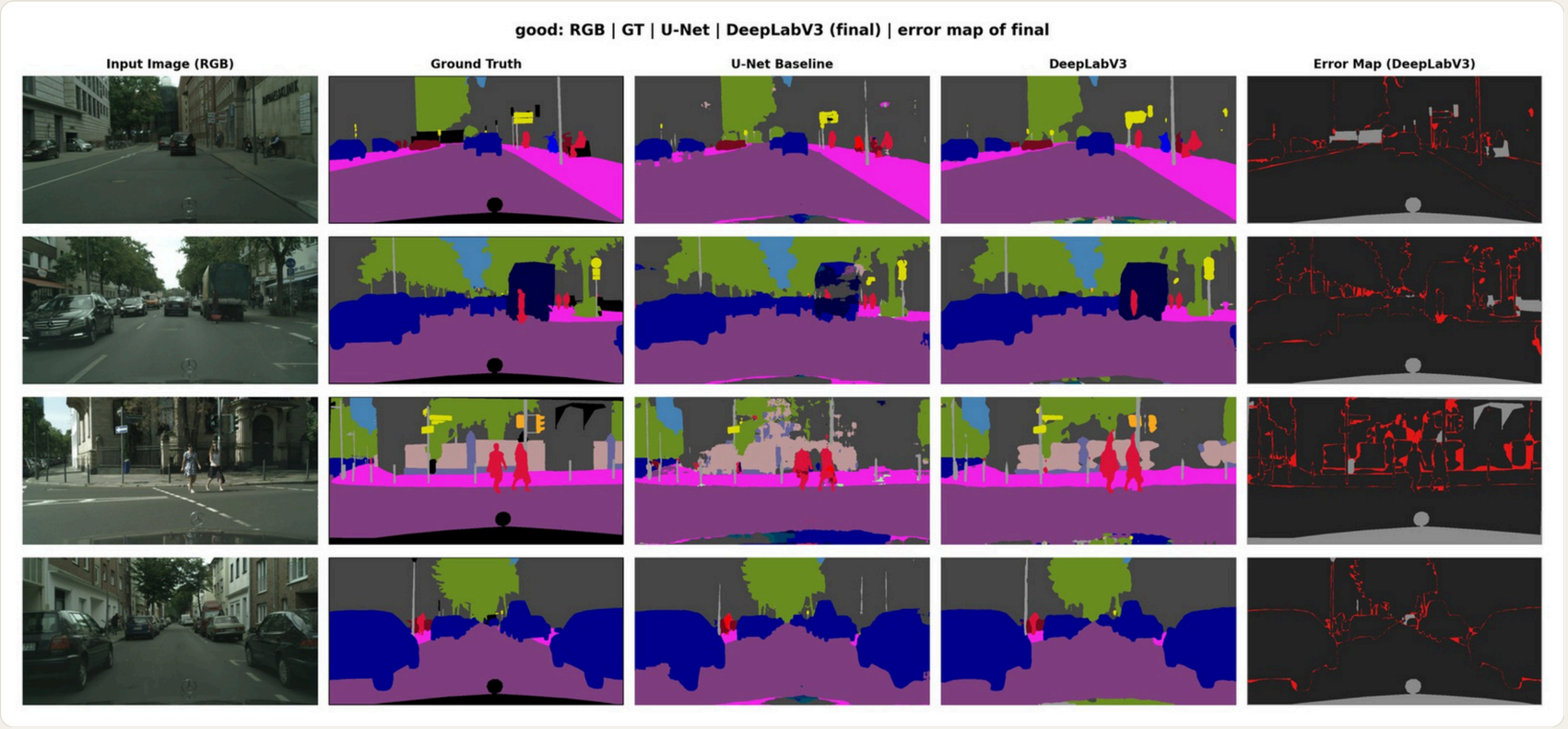


True → predicted	% of class	% of errors
building → vegetation	2.43	8.78
sidewalk → road	8.70	7.73
vegetation → building	2.42	6.90
road → sidewalk	1.02	6.29
pole → building	15.99	3.89
wall → building	26.10	3.14

For 5 of these 6 mix-ups, the worst image is from Lindau, a small town that looks unlike the training cities. Wall loses a quarter of its pixels to building.

8 · QUALITATIVE · RGB | GT | U-NET | DLV3 | ERROR MAP

Good cases: errors are thin outlines



Rule

4 test images with the highest final mIoU

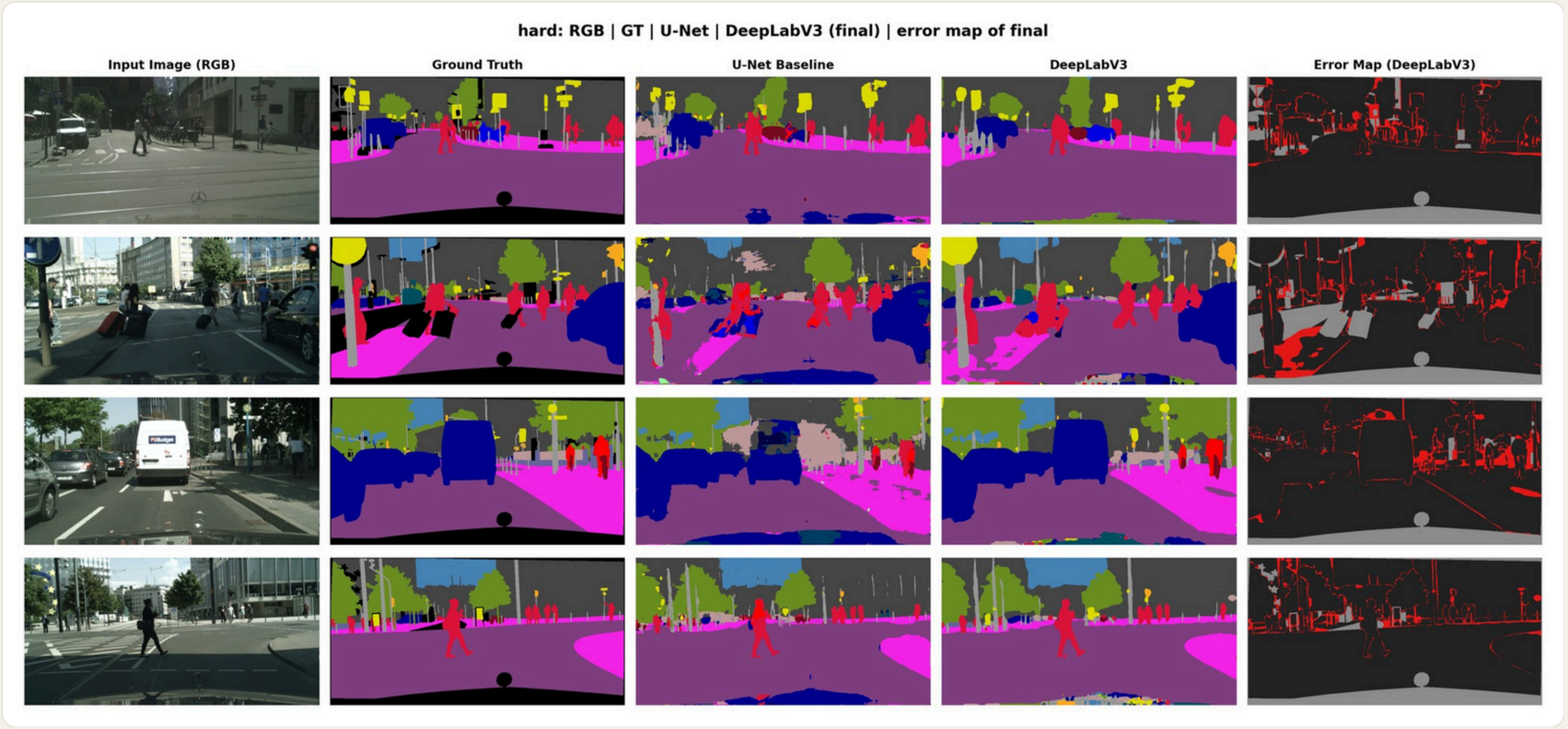
What we see

Final 0.78–0.83 mIoU vs. U-Net 0.41–0.49. Red errors follow object outlines.

munster_000012: DLV3 keeps the truck in one piece; U-Net breaks it up.

8 · QUALITATIVE · RGB | GT | U-NET | DLV3 | ERROR MAP

Hard but correct: many small objects



Rule

Top 25% by small regions (≥ 111), final mIoU above the median; the 4 largest gains over U-Net

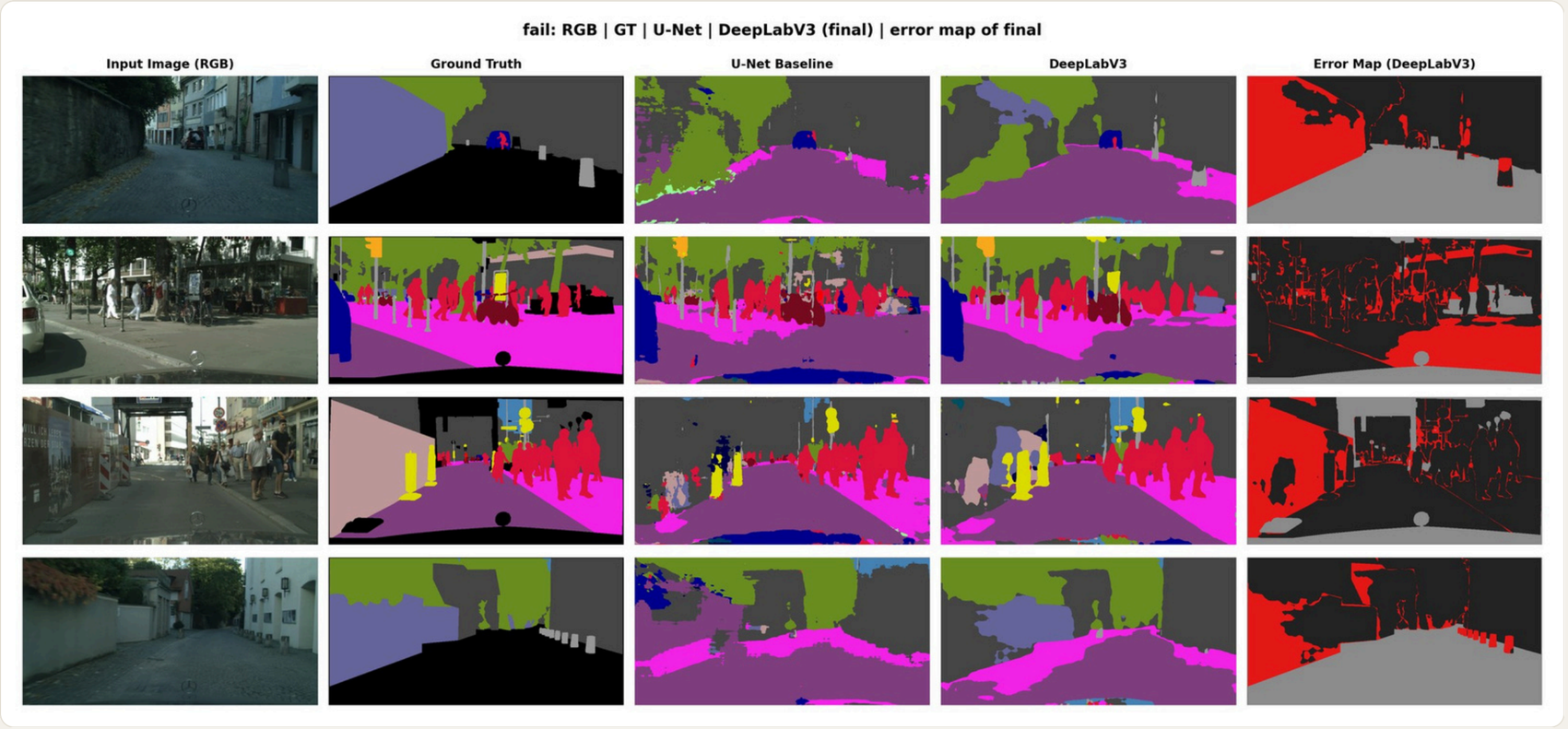
What we see

117–150 small regions each. Final 0.62–0.71 vs. U-Net 0.33–0.42 mIoU.

Poles, signs and people are separate regions; most errors are outlines.

8 · QUALITATIVE · RGB | GT | U-NET | DLV3 | ERROR MAP

Failures: walls, fences and sidewalks



Rule

4 test images with the lowest pixel accuracy (61.3–78.3%)

What we see

Lindau: a big stone wall → building or vegetation.

Frankfurt: wide sidewalk → road; construction fence → wall or building.

The same pairs as the top confusions.

9 · SHORT DEMO · REPRODUCE FROM THE REPO

From raw data to test metrics

```
# 1 · Environment (done before recording)
pip install -r requirements.txt
# 2 · Data: trainIds, 1024×512, city splits
python scripts/prepare_data.py
python scripts/pre_resize.py
python scripts/create_splits.py
# 3 · Train (show a few iterations live)
python -m src.train --config configs/unet_final.yaml
python -m src.train --config configs/dlv3_cedice.yaml
# 4 · Evaluate the selected checkpoint on test
python -m src.evaluate --config configs/dlv3_cedice.yaml \
  --checkpoint checkpoints/dlv3_cedice_1024x512_s666/best.pt \
  --split test
# 5 · Figures and error analysis
python scripts/m3_error_qual.py
```

Per-epoch log

log.csv: train loss, val loss, val mIoU, Dice, pixel accuracy, per-class IoU, time

Saved per run

best.pt (best val mIoU), last.pt, metrics_test.json

Re-check

Re-running evaluate.py on the U-Net checkpoint gives 49.35% test mIoU again

10 · CONCLUSION

Pretraining wins inside objects

Partial fine-tuning lifts test mIoU from 49.35% (U-Net) to 66.53% with CE and 68.09% with CE + Dice, at about 60 FPS. Borders stay the main error source (63% of errors).

Hypotheses (decided on validation)

Exp.	Verdict	Evidence
E0	Supported	frozen +11.28 pp, partial +19.57 pp (≥ 10 pp); whole recipe, not pretraining alone
E1	Supported	partial +8.29 pp over frozen (≥ 2 pp)
E2	Supported	Rule met by CE + Dice with +1.42pp Over CE (68.54% val mIoU),

Limitations

One seed per run; E1 on another GPU and full with a smaller LR; half resolution (not benchmark-comparable); no LR schedule; 3 daytime test cities; E1 checkpoints not uploaded yet.

Next steps

Border-aware decoder (DeepLabV3+ style), full 2048×1024 training, rare-class sampling, an LR schedule, and 3+ seeds.

DELIVERABLES & AI USAGE DISCLOSURE

Thank you — everything is reproducible

Code repository

github.com/phsuuuuuu/CO3133_sem261_groupRTX5090

Assignment page · report · slides

phsuuuuuu.github.io/CO3133_sem261_groupRTX5090/assignment2/assignment2.html

Checkpoints

[link to checkpoint folder] · or rebuild with the demo commands

Video

CO3133-Semester-261 – Group RTX5090 – Assignment 2

AI tools used (details in AI_USAGE.md)

Gemini 3.8 Flash · Cao Duc Thanh — dataset search and proposal review; data.py, U-Net and loss explanations; BatchNorm batch-1 crash; plot and error-analysis scripts; report structure (A2-1 to A2-10)

Claude Opus 5.5 · Cao Duc Thanh — checked the report against our logs, wrote the m3_* analysis scripts, drafted these slides

Claude Opus 5.5 / ChatGPT · Dang Thanh Phu — ASPP batch-size issue, ablation table format

All numbers come from our own runs, logs and metrics files.